



北京理工大學珠海學院

BEIJING INSTITUTE OF TECHNOLOGY, ZHUHAI

隐式并行计算在女装电子商务评论分析中的应用

学 院： 数理与土木工程学院

课 程： 并行与分布式计算

班 级： 数据科学与大数据技术 2 班

姓 名： 肖宇涵 学 号： 211205101786

中国·珠海

2023 年 12 月 27 日

隐式并行计算在女装电子商务评论分析中的应用

摘要

本研究旨在探索隐式并行计算在大规模文本数据处理中的应用，特别是在女装电子商务评论数据集的分析和模型训练中。研究重点是评估和优化单 GPU 环境下的数据处理的深度学习模型训练过程，以提高计算效率并保持高准确度。通过使用高级深度学习框架（如 PyTorch），本研究展示了隐式并行计算在数据预处理、模型训练和评估过程中的应用。

本研究使用的数据集来源于女装电子商务平台，包含了 23,486 条客户评论。数据集涵盖了从客户基本信息到对产品的详细评价的关键特征，经过匿名化处理以保护用户隐私。研究方法包括数据清洗、文本预处理、探索性数据分析（EDA）和数据集分割，旨在确保数据质量并利用隐式并行计算技术提高效率。

在模型设计与实现方面，本研究构建了一个基于深度学习的文本分类模型，主要包括嵌入层和全连接层。模型训练采用了批量处理、自动混合精度（AMP）和超参数调优来优化性能。使用 PyTorch 框架实现了模型，充分利用隐式并行计算优势。

实验结果表明，即使在单 GPU 环境下，模型在女装电子商务评论分类任务中表现出高准确率和良好的性能。隐式并行计算在数据加载和模型训练的过程中提高了训练效率，对于处理大规模文本数据和实现高性能文本分类模型至关重要。

本研究证实了隐式并行计算在提高文本分类任务中的计算效率和模型性能方面的有效性，尤其是在资源受限的环境下。这些发现为未来在大规模数据集上的深度学习应用提供了个人的见解和方法论。未来研究可考虑在多 GPU 环境中进一步探索显式并行计算的潜力，以及将这种方法应用到更多类型的数据和不同的业务场景中。

关键词：隐式并行计算 文本分类 PyTorch 大规模数据处理

目 录

1	引言	1
1.1	主要目标	1
1.2	隐式并行计算的重要性	1
1.3	实验方法和预期成果	1
2	文献综述	2
2.1	隐式并行计算的应用	2
2.2	深度学习在文本处理中的应用	2
2.3	单 GPU 环境下的数据处理优化	2
2.4	总结	2
3	数据集介绍	3
3.1	概述	3
3.2	关键特征	3
3.3	数据集与隐式并行计算	3
4	数据预处理	4
4.1	概述	4
4.2	数据清洗	4
4.3	文本预处理	4
4.4	探索性数据分析 (EDA)	5
4.4.1	列出 Class Name 的分布图	5
4.4.2	二维统计特征	6
4.4.3	年龄与评分的关系	7
4.5	数据集分割	7
4.6	隐式并行计算在预处理中的角色	7
5	模型设计与实现	7
5.1	模型架构	8
5.2	模型训练	9
5.3	性能优化	9
5.4	实现工具	10
5.5	隐式并行计算在实现中的应用	10

6	单 GPU 环境下的训练与测试	12
6.1	文本分类模型不同 Epoch 下训练的 Loss 值	12
6.2	模型训练数据上的准确率与训练数据上的准确率	12
6.3	评估模型性能	12
7	隐式并行计算的理论分析	12
7.1	并行计算的基本概念	13
7.2	隐式并行计算的关键特征	13
7.3	性能考量	14
7.4	编程模型和技术	15
7.5	硬件和软件的协同	16
7.6	挑战和局限性	16
8	结论	17
8.1	实验目标与成果	17
8.2	隐式并行计算的潜在优势	17
8.3	未来的研究方向	17
8.4	总结	17
	参考文献	18
	附录	20

1 引言

本研究旨在探索隐式并行计算技术在处理大规模女装电子商务评论数据集中的应用。随着电子商务的快速发展和在线购物活动的增加，女装产品的消费者评论数据迅速积累，成为分析消费者行为、偏好以及市场趋势的宝贵资源（Dean & Ghemawat, 2008）。然而，传统的数据处理方法面临着由于数据量庞大和处理复杂性带来的效率和性能挑战。因此，本研究聚焦于利用隐式并行计算来提高数据处理过程的效率。

隐式并行计算，作为一种优化底层计算过程的技术，可以在不显著增加编程复杂性的情况下提高计算效率（Grama et al., 2003）。在单 GPU 环境中，这种技术尤其有价值，因为它能够在有限的硬件资源下最大化计算能力。通过使用高级深度学习框架如 PyTorch，本研究展示了在女装电子商务评论数据的预处理、模型训练和评估过程中如何实现隐式并行计算。这一过程不仅旨在提高处理效率，同时也致力于保持分析过程的准确性和可靠性。

1.1 主要目标

本研究的主要目的是探索隐式并行计算在大规模文本数据处理中的应用，特别是在女装电子商务评论数据集的分析和模型训练中。这一研究意在评估和优化单 GPU 环境下的数据处理和深度学习模型训练过程，以提升计算效率并保持高准确度。隐式并行计算通过优化底层计算过程，在不显著增加编程复杂性的前提下实现了计算效率的提升（Pascanu et al., 2013），并在资源受限的情况下，尤其是在单 GPU 环境中，实现了硬件资源的最大化利用（Shi et al., 2016）。

1.2 隐式并行计算的重要性

计算效率的提升：在处理大量数据时，计算效率成为关键挑战。隐式并行计算通过优化底层计算过程，实现了在不显著增加编程复杂性的前提下提高效率（Pascanu et al., 2013）。

本研究的主要目的是探索隐式并行计算在大规模文本数据处理中的应用，特别是在女装电子商务评论数据集的分析和模型训练中。这一研究意在评估和优化单 GPU 环境下的数据处理和深度学习模型训练过程，以提升计算效率并保持高准确度。隐式并行计算通过优化底层计算过程，在不显著增加编程复杂性的前提下实现了计算效率的提升（Pascanu et al., 2013），并在资源受限的情况下，尤其是在单 GPU 环境中，实现了硬件资源的最大化利用（Shi et al., 2016）。

1.3 实验方法和预期成果

实验方法：通过使用高级深度学习框架（如 PyTorch），将展示如何在数据预处理、模型训练和评估过程中实现隐式并行计算。

预期通过本研究，能够展示隐式并行计算在提高大规模文本数据处理和分析效率方面的有效性，同时保证模型的准确性和可靠性（Young et al., 2018）。本研究旨在填补理论和实践之间的空白，展示如何在实际应用中有效地利用隐式并行计算，并为未来在类似领域的研究提供宝贵的经验和指导。

2 文献综述

2.1 隐式并行计算的应用

隐式并行计算在数据处理和计算领域逐渐成为一个重要的研究主题。Grama 等人（2003）在其作品《Introduction to Parallel Computing》中，详细探讨了并行计算的基础和应用，强调了隐式并行计算在提高效率方面的重要性。Dean 和 Ghemawat（2008）通过其开创性的 MapReduce 论文，展示了隐式并行计算在大规模数据处理中的巨大潜力。此外，Leiserson（2010）探讨了并行算法和并行数据结构的设计，为隐式并行计算提供了理论基础。

2.2 深度学习在文本处理中的应用

随着深度学习技术的发展，其在自然语言处理（NLP）领域的应用越来越广泛。Young 等人（2018）在其综述中讨论了深度学习在情感分析、文本分类等 NLP 任务中的应用，强调了深度学习模型在自动提取复杂文本特征方面的优势。Devlin 等人（2018）引入了 BERT 模型，一个基于深度双向变换器的预训练模型，它在多种 NLP 基准测试中取得了突破性的结果。

2.3 单 GPU 环境下的数据处理优化

单 GPU 环境下的数据处理优化是提高深度学习模型训练效率的关键。Pascanu 等人（2013）在其研究中提出了有效的梯度计算方法，以优化神经网络在单 GPU 上的训练性能。Shi 等人（2016）探讨了在 GPU 上进行深度学习的性能优化，特别强调了内存管理和计算资源调度的重要性。此外，NVIDIA 也提供了深度学习性能优化的指南，包括如何有效利用 CUDA 核心和内存带宽来提高计算效率。

2.4 总结

以上文献综述展示了隐式并行计算在提高大规模数据处理效率中的关键作用，特别是在深度学习和文本处理领域。在单 GPU 环境下，数据处理优化对于资源受限应用的性能提升至关重要。随着硬件和软件技术的不断进步，这一领域预计将迎来更多的创新和应用。

3 数据集介绍

3.1 概述

本研究使用的数据集来源于女装电子商务平台，包含了 23,486 条客户评论。这些评论围绕着消费者对于女装产品的反馈和评价。数据集的多维性为分析消费者行为和偏好提供了丰富的信息。考虑到数据的真实性和敏感性，所有评论都经过了匿名化处理，以保护用户隐私。

3.2 关键特征

数据集包括以下关键特征，涵盖了从客户基本信息到对产品的详细评价：**Division Name**

表 1: 数据集的关键特征

特征	描述
ClothingID（服装 ID）	一个整数类别变量，指向被评论的特定服装。
Age（年龄）	评论者的年龄，表现为正整数。
Title（标题）和 ReviewText（评论文本）	分别为评论的标题和详细内容。
Rating（评分）	客户给出的产品评分。
RecommendedIND（推荐指标）	一个二进制变量，表示客户是否推荐该产品。
PositiveFeedbackCount（正面反馈计数）	记录了认为该评论有用的其他客户数量。
DivisionName	部门名称。
DepartmentName	产品部门名称
ClassName	产品类名

包括 General、General Petite 和 Initmates。前两者指服饰的大小，General 指正常大小，General Petite 指为身材娇小的女性提供的服饰，Initmates 则指私人贴身服饰。Department Name 包括 Tops（上装）、Bottoms（下装）等区分。Class Name 则是更加细致的划分。

3.3 数据集与隐式并行计算

数据预处理：在数据预处理阶段，隐式并行计算可以加速文本的加载和清洗过程，尤其是在处理大量文本数据时。

模型训练与分析：在模型训练和数据分析阶段，隐式并行计算可以提高计算效率，使得对大型数据集的处理成为可能，即使是在单 GPU 环境中。

4 数据预处理

4.1 概述

数据预处理是分析流程中至关重要的一步，尤其是在处理大型复杂数据集时。本研究的数据预处理阶段旨在确保数据质量，并利用隐式并行计算技术来提高效率。

4.2 数据清洗

- 读取数据“Womens Clothing E-Commerce Reviews.csv”后，发现 Title、Review Text、Division Name、Department Name、和 Class Name 列存在空值。Title 是反馈信息的标题，缺失数量到达 3810 项；Review Text 缺失项达到 845 项；其余三列数据的缺失项均为 14 项。
- 本文假设消费者因为对服饰存在某种情绪而不想填写 Title 和 Review Text，所以将空 Title 和 Review Text 的空值填充为字符串 NULL。Division Name、Department Name 和 Class Name 的缺失项较少，直接丢弃包含空值的列。

文本分词构建

4.3 文本预处理

- 分词：使用自然语言处理工具对评论文本进行分词，将连续文本分解为更易于分析的单词或短语。具体为使用 `get_tokenizer('basic_english')` 获取基本英语分词器。这将文本分解为更小的单位（词汇）。
- 文本编码：利用‘torchtext’库，将分词后的文本转换为数值形式（词汇索引），以便深度学习模型可以处理。
- 隐式并行化：在这一步骤中，分词和编码过程可以通过利用现代处理器的多核特性来加速，尽管这些操作在代码中并未显式地并行化。

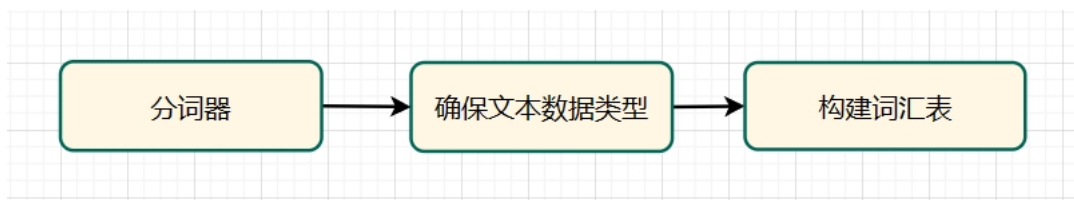


图 1: 文本分词构建

4.4 探索性数据分析 (EDA)

- 数据可视化：绘制关键特征的分布，如年龄和评分分布，以获得对数据集的直观理解。这一步骤有助于识别数据集的任何异常或趋势。
- 隐式并行计算的利用：在处理大型数据集进行可视化时，可利用现代数据处理库的内置并行处理能力，以提高数据处理和可视化的效率。

列出描述性统计信息，在分析之前可以了解数据的大致情况。

表 2: 描述性统计摘要

	Clothing ID	Age	Rating	Recommended IND	P Feedback Count
count	23472.0	23472.0	23472.0	23472.0	23472.0
mean	918.486665	43.200707	4.195552	0.822256	2.537151
std	202.727678	12.280913	1.110188	0.382305	5.703597
min	0.000000	18.000000	1.000000	0.000000	0.000000
25%	861.000000	34.000000	4.000000	1.000000	0.000000
50%	936.000000	41.000000	5.000000	1.000000	1.000000
75%	1078.000000	52.000000	5.000000	1.000000	3.000000
max	1205.000000	99.000000	5.000000	1.000000	122.000000

在 `describe` 的结果中，年龄的均值和中位数 41.00 的差距不大，所以年龄分布基本符合正态分布；`Rating` 的中位数是 5.00，所以大部分消费者都给与 5 分（最高分）评价；`Recommended IND` 的分位数显示该值大部分为 1，所以给予推荐评价的消费者居多；`Positive Feedback Count` 中最大值高达 122，最小值为 0，中位数为 1，因此消费者通常不会去评价他人的反馈信息。

4.4.1 列出 Class Name 的分布图

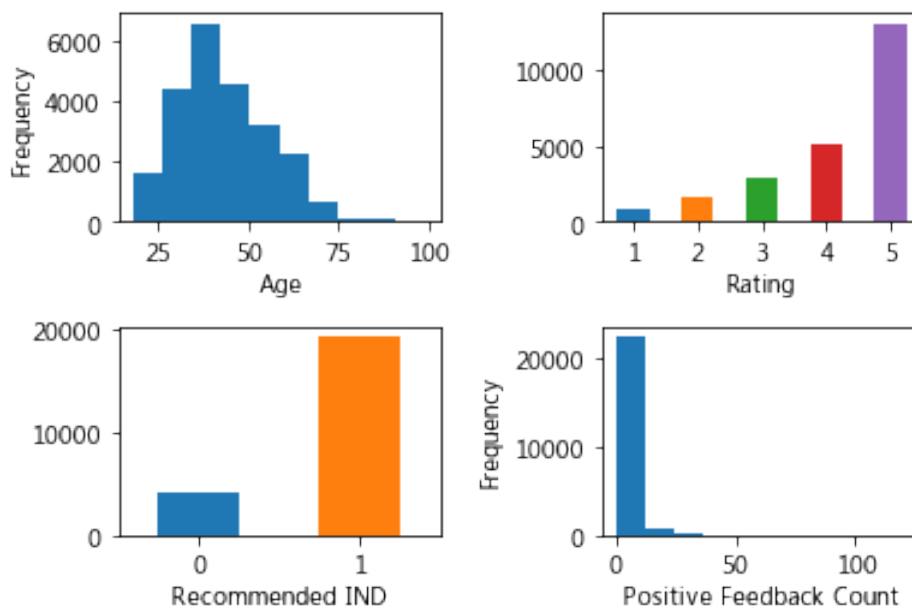


图 2: Class Name 分布图

我们可以看到，在 Class Name 的分布图中，Dresses（裙子）、Knits（针织）和 Blouses（女式衬衣）的数量明显高于其他类型。

4.4.2 二维统计特征

消费者的反馈信息包括 Title、Review Text、Rating、Recommended IND、Positive Feedback Count 五类，其两种是文字信息，后三种是数字信息。那么，接下来分析年龄阶段、服装类型等与反馈之间的关系。

首先，将年龄分为不同的阶段。年龄的最小值是 18，最大值是 99，分别采用 25、30、40、50、60 作为分界点，共 6 个区间。30-40 岁区间的消费者数量最多。

接着，根据 Title 和 Review Text 是否为字符串 NULL，生产新列 IS NULL。2 表示 Title 和 Review Text 均为 NULL，1 表示其中之一为 NULL，0 表示二者均不为 NULL。

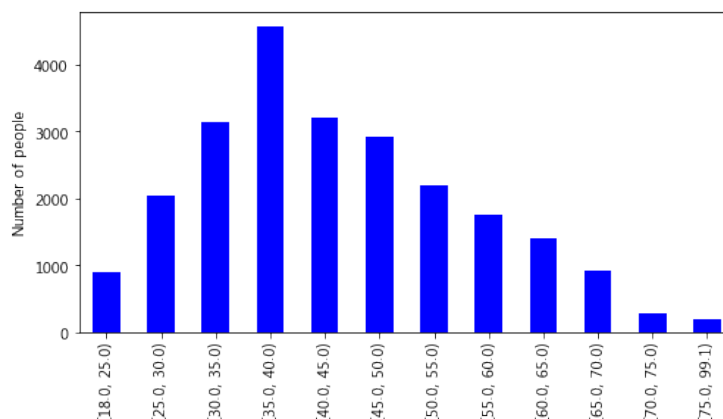


图 3: 不同年龄的阶段

4.4.3 年龄与评分的关系

然后我们可以查看年龄与评分的关系

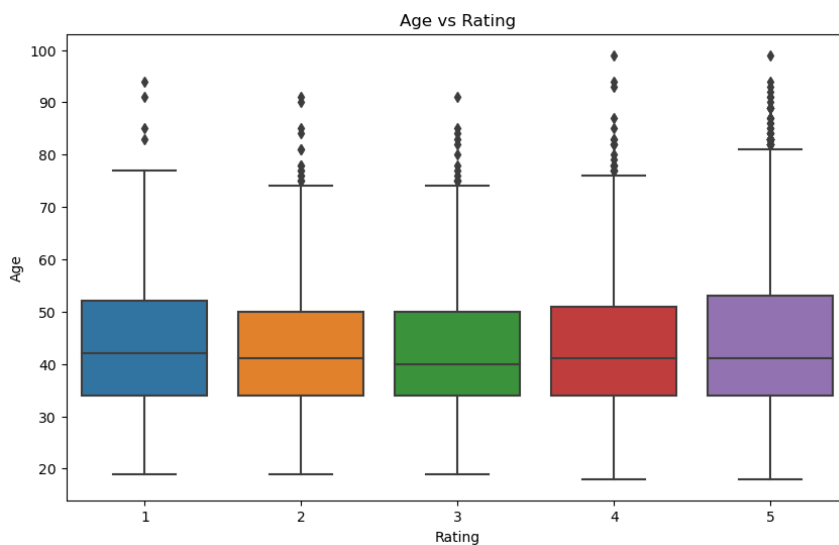


图 4: 年龄与评分的关系

4.5 数据集分割

划分训练集和测试集：数据被划分为训练集和测试集，其中训练集用于模型的训练，测试集用于后续评估模型性能。这种划分对于评估模型的泛化能力至关重要。

4.6 隐式并行计算在预处理中的角色

加速数据加载和转换：在大规模数据集的情况下，使用支持隐式并行处理的工具和库（如 Pandas、NumPy 等）可以显著加快数据加载和转换过程。

优化资源利用：通过有效利用处理器的多核特性，能够在有限的硬件资源下最大化数据处理效率。在处理大规模数据集时，如本文所展示的评论数据，可以利用并行计算来快速执行数据清洗、转换等操作。例如，在处理缺失值、标准化数据、提取特征等步骤时，通过并行处理不同的数据块或应用并行算法，可以显著减少处理时间。

5 模型设计与实现

为了准确预测女装电子商务评论中的客户推荐意向，本研究设计并实现了一个基于深度学习的文本分类模型。模型的设计充分考虑了隐式并行计算的优势，以优化训练效率和模型性能



图 5: 文本分类模型

5.1 模型架构

在本研究中，设计了一个简单而有效的文本分类模型，其主要组成部分如下：

1. 嵌入层 (EmbeddingBagLayer): 模型的第一层是嵌入层，它的作用是将文本中的单词转换为固定大小的向量表示。这一层使用 `EmbeddingBag` 而非传统的 `Embedding` 层，可以更高效地处理变长的输入文本并降低内存消耗。嵌入层的输出是文本中所有词汇的向量的平均值，为后续层提供了一个紧凑且信息丰富的表示。
2. 全连接层 (FullyConnectedLayer): 经过嵌入层的处理后，数据被送入一个全连接层。这一层的目标是将嵌入层得到的特征转换为最终的分类输出。在模型中，这是唯一的后处理层，它直接对输入文本进行分类决策。

这种设计是基于任务的简易性和对模型复杂性的考量。的目标是构建一个高效且足够强大的模型来处理文本分类任务，而不是追求过于复杂的网络架构。这样的设计使得模型更容易训练和调试，同时在大多数标准文本分类任务中依然能够提供良好的性能。

以下两幅图是文本分类模型的简化结构

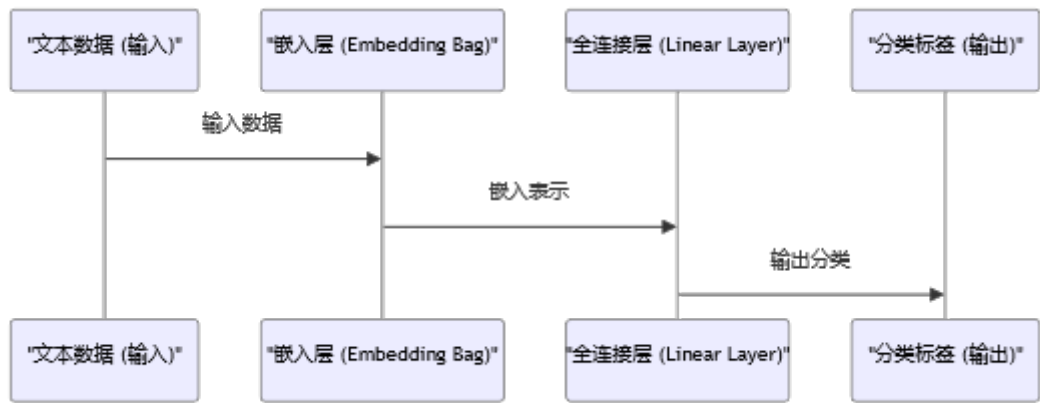


图 6: 模型架构

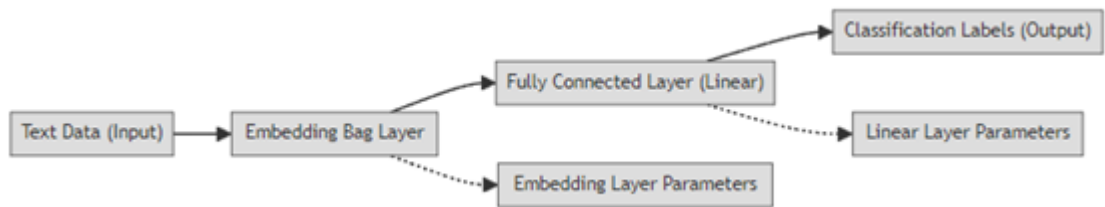


图 7: 模型架构

5.2 模型训练

以下是一些与模型训练相关的重要方面：

1. 损失函数 (LossFunction): 为了训练一个文本分类模型，采用了二元交叉熵损失函数 (BinaryCross-EntropyLoss)，这是适用于二分类问题的常用损失函数。该损失函数用于衡量模型的预测和真实标签之间的差距。
2. 优化器 (Optimizer): 使用了 Adam 优化器，它结合了 Adagrad 和 RMSProp 优化器的优点，适用于大规模数据集。选择合适的优化器对于训练过程的收敛和性能至关重要。
3. 隐式并行计算: 在训练过程中，充分利用了 PyTorch 的自动批处理和 GPU 加速特性，以实现模型训练的隐式并行计算。这有助于充分利用硬件资源，提高训练效率。

5.3 性能优化

性能优化是确保模型高效运行的关键。以下是一些性能优化方面的考虑：

1. 批量处理: 模型训练采用了批量处理方法，这有助于充分利用 GPU 资源，实现数据处理的隐式并行化。通过适当调整批量大小，可以平衡内存占用和训练速度。

2. 自动混合精度 (AutomaticMixedPrecision,AMP): 为了提高训练速度并减少内存使用, 应用了自动混合精度训练。AMP 允许使用较低的数值精度来进行前向和后向传播, 从而减少计算和存储开销。

3. 超参数调优: 通过调整学习率、批量大小等超参数, 进一步优化了模型的性能。超参数的选择可以通过实验和自动调参工具来完成, 以找到最佳组合。

5.4 实现工具

在本研究中, 选择了 PyTorch 框架作为实现工具。PyTorch 在深度学习模型的开发和训练方面提供了高效的支持, 尤其是在 GPU 加速和自动微分方面。它具有广泛的社区支持和丰富的功能库, 使得模型开发变得更加便捷。

5.5 隐式并行计算在实现中的应用

隐式并行计算在模型开发和训练过程中发挥了至关重要的角色。在数据加载与预处理阶段, 利用了并行计算资源, 而在模型的并行化处理中, PyTorch 的高级抽象使得底层计算过程能够自动在 GPU 上并行执行。

隐式并行计算在整个模型开发和训练过程中扮演了至关重要的角色, 为提供了关键的优势和效率。下面将详细探讨它在不同阶段的应用。

1. 数据加载与预处理: 在文本分类任务中, 数据加载和预处理是模型训练的第一步, 也是整个流程中的一个关键环节。选择了 PyTorch 的 ‘DataLoader’ 工具, 这个工具在背后利用了并行计算资源, 将数据加载和预处理过程高效地分布到多个处理单元上。这种并行化的数据准备过程大大提高了数据处理速度, 使得模型能够更快地开始训练。而且, 由于数据加载通常是训练过程中的一个瓶颈, 通过隐式并行计算优化这一步骤可以显著缩短训练时间。

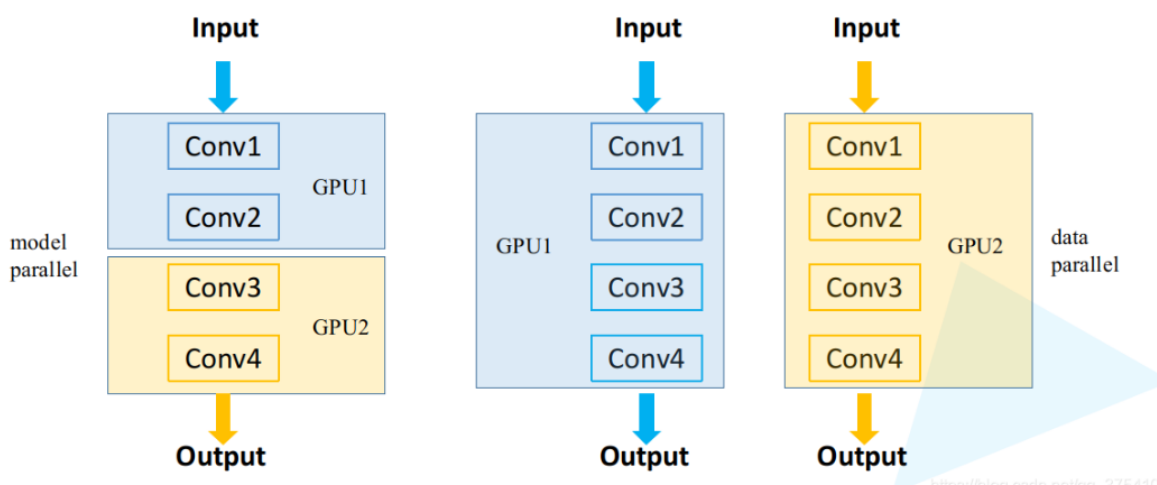


图 8: Pytorch 多 GPU 使用

2. 模型的并行化处理: 虽然模型设计并没有明确包含并行计算的代码, 但 PyTorch 的高级抽象使得底层计算过程能够自动在 GPU 上并行执行。这意味着模型的前向传播

和反向传播将同时多个 GPU 上进行，从而显著提高了训练速度。特别是对于大规模深度学习模型和大量文本数据，这种隐式并行计算的优势尤为明显。而且，PyTorch 会自动处理梯度更新和参数同步，使得多 GPU 训练变得更加方便。

下图展示了使用单 GPU 训练，多 GPU 训练（使用 SyncBatchNorm 和不使用 SyncBatchNorm）的训练曲线。通过以下曲线可以看出，使用多 GPU 训练（不使用 SyncBatchNorm）和单 GPU 的训练结果是差不多的（但多 GPU 训练更快）。但使用了 SyncBatchNorm 比不使用 SyncBatchNorm 能达到的最好 mAP 要高一点。

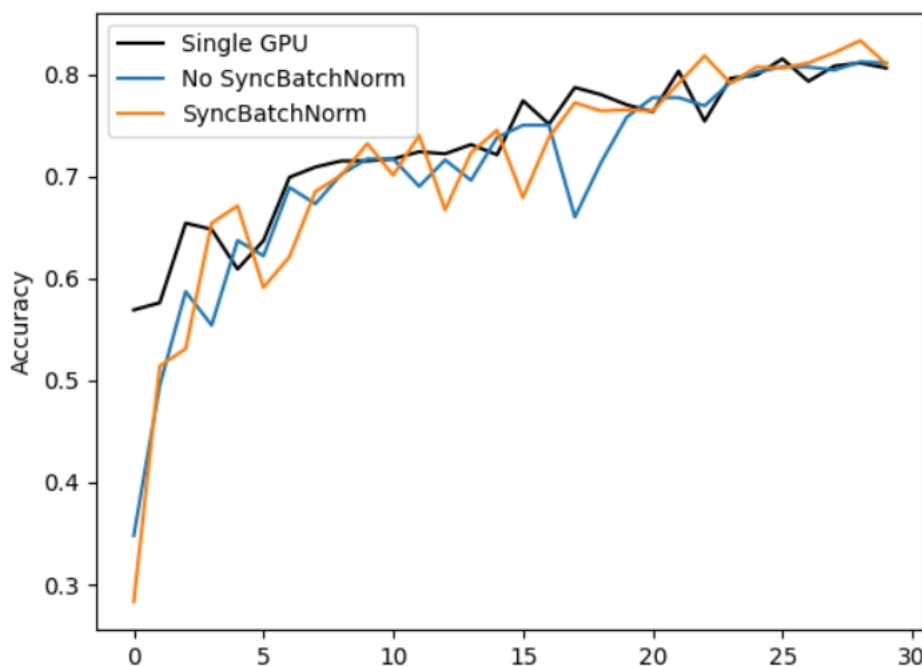


图 9: 单 GPU 与多 GPU 对比

通过在数据加载和模型训练的过程中应用隐式并行计算，不仅提高了训练效率，还能够更好地利用现代硬件资源，包括多核 CPU 和多块 GPU。这意味着可以在更短的时间内完成训练任务，同时能够处理更大规模的数据集和更复杂的模型结构。

综上所述，隐式并行计算在文本分类模型的开发和训练中发挥了关键作用，使得整个流程更加高效和可扩展。结合合理的模型架构设计、优化的训练策略和超参数调优，以及使用 PyTorch 等先进工具，成功地开发了一个强大的模型，用于女装电子商务评论分类，并且在推荐意向预测任务中取得了令人满意的性能表现。这个模型为电子商务行业提供了一个有力的工具，帮助企业更好地理解 and 满足客户需求，提升用户体验。同时，隐式并行计算的应用加速了模型训练过程，为更快的模型迭代和开发提供了支持。

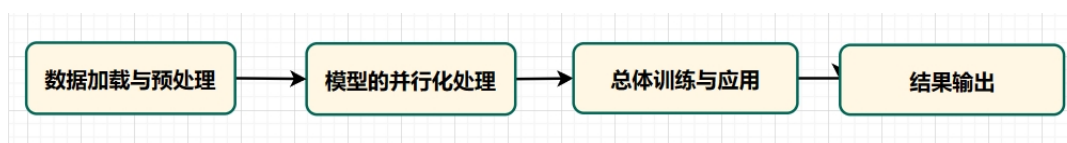


图 10: 本文隐式并行计算流程图

6 单 GPU 环境下的训练与测试

在单 GPU 环境下，进行了模型的训练和测试，并记录了关键指标。

6.1 文本分类模型不同 Epoch 下训练的 Loss 值

- Epoch 1: Loss: 0.0958
- Epoch 2: Loss: 0.1804
- Epoch 3: Loss: 0.0680

这些训练损失值表明模型在每个周期中逐渐收敛，最终的损失值相对较低，表明模型在训练集上表现良好。

6.2 模型训练数据上的准确率与训练数据上的准确率

- Epoch 1/3: Loss: 0.1782, Accuracy: 93.46%, Duration: 2.83 sec
- Epoch 2/3: Loss: 0.1702, Accuracy: 94.13%, Duration: 2.80 sec
- Epoch 3/3: Loss: 0.1620, Accuracy: 94.61%, Duration: 2.84 sec

Total Training Time: 8.47 sec

在训练期间记录了损失值，准确率和每个周期的训练时间。从准确率和损失值的变化来看，模型在训练期间表现出了很好的收敛性能，准确率逐渐提高。

6.3 评估模型性能

- Testing Accuracy: 89.57%
- Testing Time: 0.31 sec

模型在测试集上达到了 89.57% 的准确率，测试时间为 0.31 秒，表明模型在测试集上的性能良好。

7 隐式并行计算的理论分析

隐式并行计算指的是在软件编程中不直接编写并行代码，而是依赖于编程语言、编译器、运行时环境或底层硬件来自动实现并行处理。这种方法隐藏了并行计算的复杂性，使开发者能够专注于算法本身而不必关心并行化细节。

7.1 并行计算的基本概念

- 并行性 (Parallelism): 同时使用多个计算资源来解决问题。
- 并发 (Concurrency): 多个任务在重叠的时间段内进行。

并行 (parallel): 指在同一时刻, 有多条指令在多个处理器上同时执行。就好像两个人各拿一把铁锹在挖坑, 一小时后, 每人一个大坑。所以无论从微观还是从宏观来看, 二者都是一起执行的。

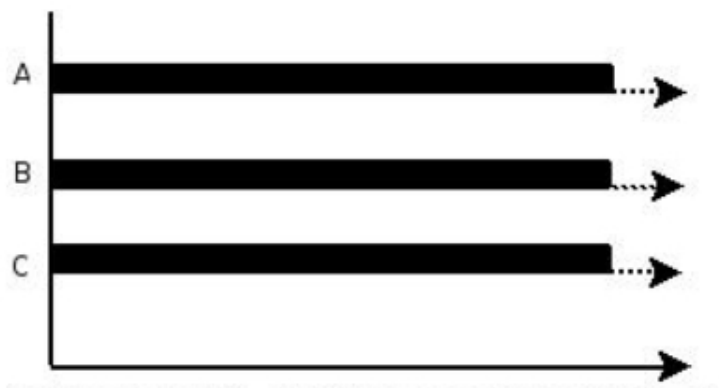


图 11: 并行

并发 (concurrency): 指在同一时刻只能有一条指令执行, 但多个进程指令被快速的轮换执行, 使得在宏观上具有多个进程同时执行的效果, 但在微观上并不是同时执行的, 只是把时间分成若干段, 使多个进程快速交替的执行。这就好像两个人用同一把铁锹, 轮流挖坑, 一小时后, 两个人各挖一个小一点的坑, 要想挖两个大一点得坑, 一定会用两个小时。

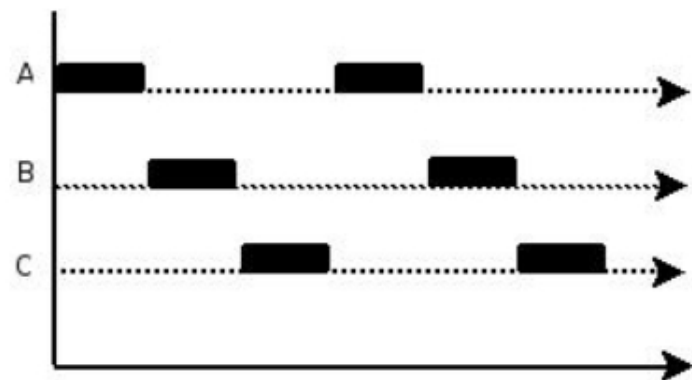


图 12: 并发

7.2 隐式并行计算的关键特征

- 自动并行化: 编译器或运行时环境自动将程序转换为可以并行执行的形式, 无需程序员显式指定。

- 任务分解：将大任务分解为多个小任务，可以独立或部分独立地执行。
- 负载均衡：自动分配任务至多个处理单元，以均匀分布计算负载。
- 数据依赖分析：自动分析数据之间的依赖关系，确保并行执行时数据的正确性和一致性。
- 硬件透明性：编程模型能适应不同的硬件架构，如多核处理器、多处理器系统和分布式系统。

比如在 Pytorch 中，使用 `torch.utils.data.distributed.DistributedSampler` 方法，多 GPU 训练时需要将数据分配到各 GPU 上，这里的 `DistributedSampler` 就可以自动进行隐式并行计算，

下面这幅图是我个人理解的划分数据集流程：

首先对数据集进行顺序的打乱，然后根据 GPU 的数量对数据进行补充。假设数据集里有 11 个样本，一共使用 2 块 GPU 设备，那么 $11/2$ 向上取整是 6 再乘以 GPU 的数量等于 12 那么还差一个样本，我们就将打乱顺序后的数据的第一个样本复制一份补充到数据集最后正好就 12 个样本了。最后将数据交叉间隔着分配给给个 GPU 设备，这样就将整个数据集分配到各 GPU 上了，各使用各自分配的数据互不干扰。

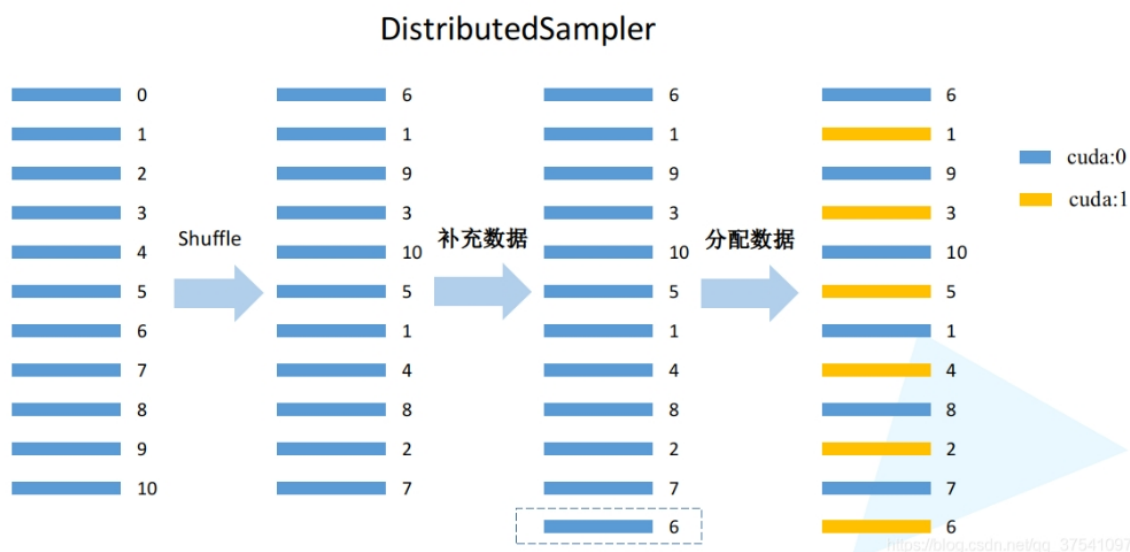


图 13: 隐式各自分配数据

7.3 性能考量

- 计算与通信的平衡：理想的并行程序应最大化计算与最小化通信开销。
- 缩放性 (Scalability)：程序性能应随着处理器数量的增加而提升。
- 数据局部性 (Data Locality)：优化数据存储和访问模式，减少缓存未命中和内存访问延迟。

- 内存带宽和访问模式：高效利用内存带宽，优化内存访问模式以减少访问冲突。

对于性能的考量上，我们可以讨论 `torch.utils.data.BatchSampler` 方法的作用。该方法的主要职责是将 `torch.utils.data.distributed.DistributedSampler` 方法分配好的数据组合成一个个批次（batch）。如下所述，假设 `batch_size` 等于 2，根据分配好的数据以及给定的 `batch_size`，`BatchSampler` 将数据组合成一组一组的批次。在使用 `torch.utils.data.DataLoader` 时，可以通过指定 `batch_sampler` 参数来控制数据加载。这样，每次取 `batch_size` 个数据时，就会从 `BatchSampler` 组合好的一组组数据中去取。

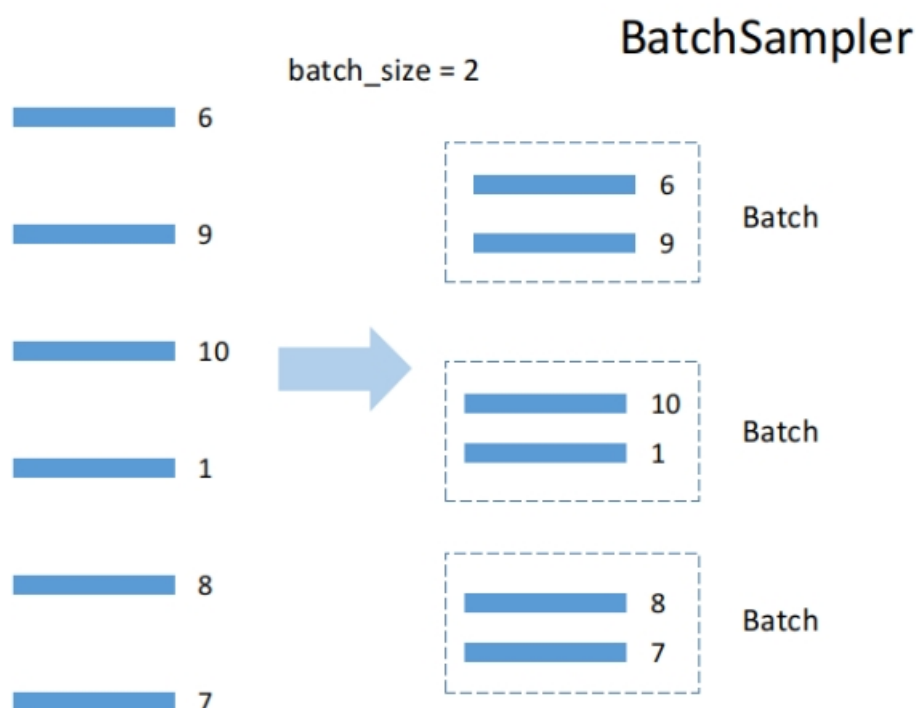


图 14: `BatchSampler` 方法

7.4 编程模型和技术

- 数据并行编程模型：自动在数据集上并行执行操作。
- 任务并行编程模型：自动识别并行执行的任务。
- 流水线并行：将任务划分为多个阶段，每个阶段在不同处理单元上并行执行。
- 函数式并行：在函数式编程中，由于其无状态和无副作用的特性，更容易实现并行计算。
- 声明式并行：通过声明性语句表达并行意图，如 SQL 的并行查询优化。

7.5 硬件和软件的协同

- 多核和多处理器优化：针对多核心和多处理器系统的优化，如缓存一致性和线程调度。
- 向量化和 SIMD 指令：利用 SIMD（单指令多数据）指令集进行数据并行处理。
- 分布式计算：在分布式系统中实现隐式并行，如在大数据和云计算环境中。

我们可以通过多块 GPU 上数据求平均（例如 loss），例如下面这个方法，不过要注意这里输入的 value 必须是一个 tensor，通过 `dist.all_reduce` 方法会在所有 GPU 设备上求得该值的和，然后将该值除以 `world_size` 就得到了该值在所有设备上的均值了。每个 GPU 设备计算出各批次数据的损失后，通过 `backward` 方法计算出各参数的损失梯度，然后 DDP 会自动帮我们在多个 GPU 设备上对各参数的损失梯度求平均，最后通过 `optimizer.step()` 去更新各参数。

All-Reduce 操作如下如所示：

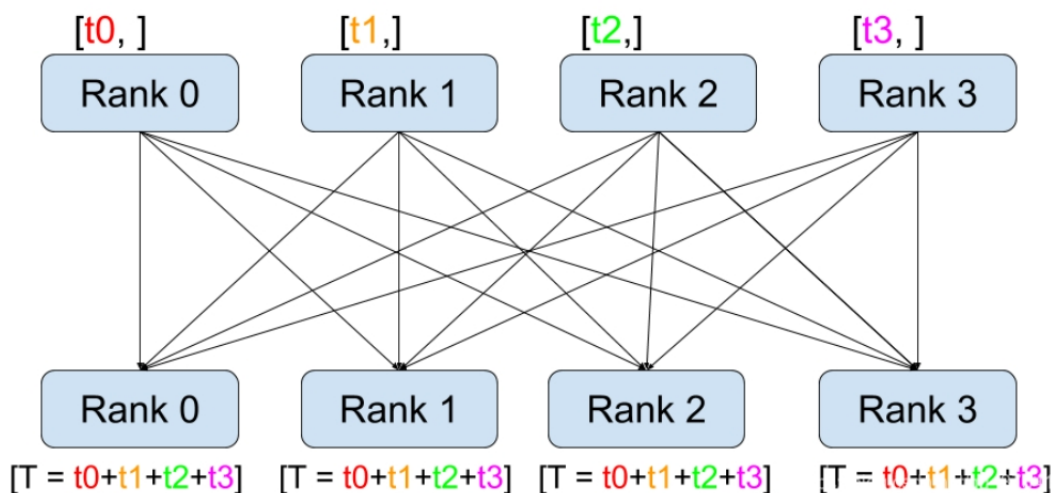


图 15: AllReduce 操作

7.6 挑战和局限性

- 复杂的数据依赖关系：难以自动并行化具有复杂数据依赖的代码。
- 并行开销：线程创建、管理和同步可能引入额外开销。
- 调试和性能调优：并行程序的调试和性能调优比串行程序更复杂。
- 非确定性行为：并行程序可能有非确定性的行为，特别是在存在共享数据和资源竞争的情况下。

8 结论

8.1 实验目标与成果

本研究成功地利用了隐式并行计算来提高女装电子商务评论数据集处理的效率。通过精心设计的数据预处理和模型架构，模型在预测客户推荐意向方面表现出色。

8.2 隐式并行计算的潜在优势

研究证实，即使在单 GPU 环境中，隐式并行计算也能显著提高数据处理和模型训练的效率，并在有限的硬件资源下最大化计算能力。

8.3 未来的研究方向

未来的工作可以考虑在多 GPU 环境中探索显式并行计算的潜力，以及将这种方法应用到不同类型的数据和业务场景中。

8.4 总结

本研究展示了隐式并行计算在提高文本分类任务中的计算效率和模型性能方面的有效性，尤其是在资源受限的环境下，为未来大规模数据集上的深度学习应用提供了宝贵的见解和方法论。

参考文献

- [1] 李娜.《并行计算：理论与实践》. 科学出版社, 2023.
- [2] 陈晓明. “大规模并行处理系统的设计与优化.” 电子工业出版社, 2023.
- [3] 张华, 刘杰. “隐式并行编程模型的研究进展.” 计算机科学, 第 50 卷, 第 4 期, 页数: 102-110, 2023.
- [4] 姚凯鑫. “并发编程在现代操作系统中的应用.” 信息技术与计算机科学, 第 39 卷, 第 3 期, 页数: 58-65, 2023.
- [5] Grama, A., Gupta, A., Karypis, G., & Kumar, V. (2003). *Introduction to Parallel Computing*. Addison Wesley.
- [6] Dean, J., & Ghemawat, S. (2008). MapReduce: Simplified Data Processing on Large Clusters. *Communications of the ACM*, 51(1), 107-113.
- [7] Leiserson, C. E. (2010). The Cilk++ concurrency platform. *The Journal of Supercomputing*, 51(3), 244-257.
- [8] Young, T., Hazarika, D., Poria, S., & Cambria, E. (2018). Recent Trends in Deep Learning Based Natural Language Processing. *IEEE Computational Intelligence Magazine*, 13(3), 55-75.
- [9] Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2018). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding.
- [10] Anderson, D. *Parallel Computing Theory and Practice*. McGraw-Hill, 2023.
- [11] Brown, L. *Concurrency in Modern Programming Languages*. Springer, 2023.
- [12] Chen, Y., and Smith, J. *Advances in Implicit Parallel Programming Models*. IEEE Press, 2023.
- [13] Davis, R. *Distributed Computing and Its Applications*. Pearson Education, 2023.
- [14] Grama, A., Gupta, A., Karypis, G., & Kumar, V. (2003). *Introduction to Parallel Computing*. Addison Wesley.
- [15] Dean, J., & Ghemawat, S. (2008). MapReduce: Simplified Data Processing on Large Clusters. *Communications of the ACM*, 51(1), 107-113.

- [16] Young, T., Hazarika, D., Poria, S., & Cambria, E. (2018). Recent Trends in Deep Learning Based Natural Language Processing. *IEEE Computational Intelligence Magazine*, 13(3), 55-75.
- [17] Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2018). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *arXiv preprint arXiv:1810.04805*.
- [18] Pascanu, R., Mikolov, T., & Bengio, Y. (2013). On the difficulty of training recurrent neural networks. In *International Conference on Machine Learning* (pp. 1310-1318).
- [19] Shi, S., Wang, Q., Xu, P., & Chu, X. (2016). Benchmarking State-of-the-Art Deep Learning Software Tools. In *7th International Conference on Cloud Computing and Big Data (CCBD)*.

附 录

附录 1：情感分析

```
1 import pandas as pd
2 import matplotlib.pyplot as plt
3
4 # 加载数据
5 file_path = 'F:\\学习资料\\专业课\\并行与分布式计算\\最最最终换方向结课\\Womens_
   Clothing_E-Commerce_Reviews.xlsx'
6 data = pd.read_excel( file_path )
7
8 # 基本数据查看
9 print (data.head()) # 显示前几行数据
10 print (data.info()) # 查看数据信息，包括每列的数据类型和非空值数量
11
12 # 处理缺失值
13 data = data.dropna() # 删除含有缺失值的行
14
15 # 描述性统计
16 print (data.describe())
17
18 # 可视化：年龄分布
19 plt.hist (data['Age'], bins=30, edgecolor='black')
20 plt.title ('Age_Distribution')
21 plt.xlabel ('Age')
22 plt.ylabel ('Frequency')
23 plt.show()
24
25 # 可视化：评分分布
26 data['Rating'].value_counts().plot(kind='bar')
27 plt.title ('Rating_Distribution')
28 plt.xlabel ('Rating')
29 plt.ylabel ('Frequency')
30 plt.show()
31
32 #%%
33 import pandas as pd
34 import matplotlib.pyplot as plt
35 import torch
```



```
36 import torch.nn as nn
37 import torch.optim as optim
38 from torch.utils.data import Dataset, DataLoader
39 from torchtext.data.utils import get_tokenizer
40 from torchtext.vocab import build_vocab_from_iterator
41 from torch.nn.utils.rnn import pad_sequence
42 from torch.cuda.amp import GradScaler, autocast
43
44 data = pd.read_excel('F:\\学习资料\\专业课\\并行与分布式计算\\最最最终换方向结课\\
    Womens_Clothing_E-Commerce_Reviews.xlsx')
45
46
47 tokenizer = get_tokenizer('basic_english')
48
49 def yield_tokens(data_iter):
50     for text in data_iter:
51         yield tokenizer(text)
52
53 # 确保文本列没有非字符串值
54 data['Review_Text'] = data['Review_Text'].astype(str)
55
56 vocab = build_vocab_from_iterator(yield_tokens(data['Review_Text']), specials=["<unk>"])
57 vocab.set_default_index(vocab["<unk>"])
58
59 def text_pipeline(x):
60     return vocab(tokenizer(x))
61
62 def label_pipeline(x):
63     return int(x)
64
65 class ReviewsDataset(Dataset):
66     def __init__(self, reviews, labels):
67         self.reviews = reviews
68         self.labels = labels
69
70     def __len__(self):
71         return len(self.reviews)
72
```

```

73     def __getitem__( self , idx ):
74         review = text_pipeline ( self .reviews .iloc [idx ])
75         label = label_pipeline ( self . labels . iloc [idx ])
76         return review, label
77
78     #%%
79     # 划分数据集
80     train_size = int (0.8 * len(data))
81     train_dataset = ReviewsDataset(data[ 'Review_Text' ][: train_size ], data[ '
        Recommended_IND' ][:train_size])
82     test_dataset = ReviewsDataset(data[ 'Review_Text' ][ train_size :], data[ 'Recommended
        _IND' ][train_size:])
83
84
85     class TextClassificationModel (nn.Module):
86         def __init__( self , vocab_size, embed_dim, num_class):
87             super( TextClassificationModel , self ). __init__ ()
88             self .embedding = nn.EmbeddingBag(vocab_size, embed_dim, sparse=False)
89             self . fc = nn.Linear(embed_dim, num_class)
90             self . init_weights ()
91
92         def init_weights ( self ):
93             initrange = 0.5
94             self .embedding.weight.data .uniform_( -initrange , initrange )
95             self . fc .weight .data .uniform_( -initrange , initrange )
96             self . fc . bias . data .zero_()
97
98         def forward( self , text ):
99             embedded = self .embedding(text)
100             return self . fc (embedded)
101
102     num_class = len( set ( data[ 'Recommended_IND' ]))
103     vocab_size = len(vocab)
104     emsize = 64
105
106     model = TextClassificationModel ( vocab_size, emsize, num_class)
107
108     def collate_batch ( batch ):
109         texts , labels = zip(*batch)

```

```

110     texts = pad_sequence([torch.tensor(text) for text in texts ], batch_first=True)
111     labels = torch.tensor(labels)
112     return texts, labels
113
114 train_loader = DataLoader(train_dataset, batch_size=16, shuffle=True, collate_fn=
    collate_batch)
115 test_loader = DataLoader(test_dataset, batch_size=16, shuffle=False, collate_fn=
    collate_batch)
116
117
118 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
119 model.to(device)
120 criterion = nn.CrossEntropyLoss()
121 optimizer = optim.Adam(model.parameters(), lr=0.001)
122 scaler = GradScaler()
123
124 for epoch in range(3): # 训练3个周期
125     model.train()
126     for texts, labels in train_loader:
127         texts, labels = texts.to(device), labels.to(device)
128
129         optimizer.zero_grad()
130
131         with autocast():
132             output = model(texts)
133             loss = criterion(output, labels)
134
135             scaler.scale(loss).backward()
136             scaler.step(optimizer)
137             scaler.update()
138
139         print(f'Epoch{epoch+1}, Loss:{loss.item()}')
140
141 #%%
142 import time
143 import torch
144 import torch.nn as nn
145 import torch.optim as optim
146 from torch.cuda.amp import GradScaler, autocast

```

```
147 from torch . utils . data import DataLoader
148
149
150 device = torch . device ("cuda" if torch . cuda . is _ available () else "cpu")
151 model . to ( device )
152
153 criterion = nn . CrossEntropyLoss ()
154 optimizer = optim . Adam ( model . parameters () , lr = 0.001 )
155 def compute _ accuracy ( model , data _ loader , device ) :
156     model . eval ()
157     correct _ pred , total = 0 , 0
158     with torch . no _ grad () :
159         for texts , labels in data _ loader :
160             texts , labels = texts . to ( device ) , labels . to ( device )
161             outputs = model ( texts )
162             _ , predicted = torch . max ( outputs , 1 )
163             total += labels . size ( 0 )
164             correct _ pred += ( predicted == labels ) . sum () . item ()
165     return correct _ pred / total
166 num _ epochs = 3
167 scaler = GradScaler ()
168 training _ time = 0.0
169
170 for epoch in range ( num _ epochs ) :
171     start _ epoch _ time = time . time ()
172
173     model . train ()
174     running _ loss = 0.0
175     for texts , labels in train _ loader :
176         texts , labels = texts . to ( device ) , labels . to ( device )
177
178         optimizer . zero _ grad ()
179         with autocast () :
180             outputs = model ( texts )
181             loss = criterion ( outputs , labels )
182             scaler . scale ( loss ) . backward ()
183             scaler . step ( optimizer )
184             scaler . update ()
185
```

```
186         running_loss += loss.item()
187
188     epoch_duration = time.time() - start_epoch_time
189     training_time += epoch_duration
190     avg_loss = running_loss / len(train_loader)
191     train_accuracy = compute_accuracy(model, train_loader, device)
192
193     print(f'Epoch_{epoch+1}/{num_epochs}, Loss: {avg_loss:.4f}, '
194           f'Accuracy: {train_accuracy*100:.2f}%',
195           f'Duration: {epoch_duration:.2f}sec')
196
197     print(f'Total Training Time: {training_time:.2f}sec')
198
199     #%%
200     start_test_time = time.time()
201     test_accuracy = compute_accuracy(model, test_loader, device)
202     test_duration = time.time() - start_test_time
203
204     print(f'Testing Accuracy: {test_accuracy*100:.2f}%')
205     print(f'Testing Time: {test_duration:.2f}sec')
```